SYED SANIYA IQBAL(192465043)

MANDA KRUTHIKA REDDY (192465037)

CO1 AT1 PART-2

Team 1 – Digital Banking Platform

Background

SecureBank Ltd. operates a nationwide digital banking platform serving more than fivemillion customers. The bank offers online banking, mobile banking, digital payments, loanprocessing and investment services. To remain competitive with fintech companies, management mandates weekly software releases.

The development team commits directly to the main branch, code reviews are optional,security testing is performed only before production deployment, third-party libraries areadopted without verification, password policies are weak, and API authentication isinconsistent. Application logs are generated but rarely reviewed. Developers have not received secure coding training.Recently, customers reported unauthorized login attempts and suspicious transactions.Internal investigation revealed exposed API keys, delayed security patches and outdatedlibraries.

Investigation Questions

1. Identify at least five security risks.

2. Identify at least five vulnerabilities and classify each as High, Medium or Low.

3. List possible cyber threats and explain their impact.

4. Prepare a DevOps vs DevSecOps comparison table (minimum eight points).

5. Recommend five OWASP security best practices.

6. Explain the business impact if these issues remain unresolved.

7. Create a concept map linking Risk → Vulnerability → DevSecOps Practice → BusinessBenefit.

8. Prepare a five-minute presentation summarizing your findings.

Abstract

Modern banking systems process millions of financial transactions every day, making them attractive targets for cybercriminals. SecureBank Ltd. has adopted rapid software releases to remain competitive; however, security practices have not evolved alongside development speed. The organization currently suffers from weak software development practices, poor security governance, inadequate authentication mechanisms, outdated third-party components, delayed patch management, and insufficient developer security awareness.

This report analyzes the organization's current DevOps process, identifies security risks and vulnerabilities, evaluates potential cyber threats, recommends OWASP security best practices, compares DevOps with DevSecOps, assesses business impacts, and proposes a secure software development strategy for SecureBank Ltd.

1. Introduction

DevSecOps integrates security into every stage of the Software Development Life Cycle (SDLC). Instead of treating security as the final testing phase, DevSecOps makes security a continuous responsibility shared among developers, operations teams, and security professionals.

SecureBank Ltd. currently releases software weekly to satisfy business demands. However, rapid development without integrated security controls has resulted in exposed API keys, outdated software libraries, unauthorized login attempts, suspicious banking transactions, delayed patching, and inconsistent API authentication.

The objective of this report is to evaluate the current development process and recommend improvements based on DevSecOps and OWASP principles.

2. Existing Process Review

Current Development Process Weaknesses

No	Existing Weakness	Technical Explanation	Impact
1	Developers commit directly to the Main branch	No branch protection or pull request validation	High probability of introducing insecure code
2	Code reviews are optional	Security flaws remain undetected	Poor code quality and increased vulnerabilities
3	Security testing occurs only before deployment	Late detection of vulnerabilities	Expensive fixes and delayed releases
4	Third-party libraries are used without verification	Libraries may contain malware or known CVEs	Supply chain attacks
5	Weak password policy	Simple passwords are easier to crack	Unauthorized access
6	Inconsistent API authentication	Some APIs lack proper authentication	Unauthorized API access
7	Application logs are rarely reviewed	Security incidents remain unnoticed	Delayed incident response
8	Developers lack secure coding training	Common coding mistakes increase	OWASP Top 10 vulnerabilities
9	Delayed security patch installation	Known vulnerabilities remain exploitable	Higher attack surface
10	Outdated software libraries	Contain publicly known vulnerabilities	Remote code execution risk

3. Security Risk Identification

Identified Security Risks

Risk 1: Unauthorized Account Access

Weak passwords and inconsistent authentication enable attackers to gain unauthorized access to customer accounts.

Likelihood: High

Impact: High

Risk 2: API Key Exposure

Exposed API keys allow attackers to directly access backend services.

Likelihood: High

Impact: Critical

Risk 3: Supply Chain Attack

Using third-party libraries without verification increases the risk of malicious dependencies.

Likelihood: Medium

Impact: High

Risk 4: Delayed Vulnerability Remediation

Security patches are not applied immediately.

Attackers exploit known vulnerabilities before patches are installed.

Risk 5: Insider Threat

Developers with unrestricted repository access may accidentally or intentionally introduce vulnerabilities.

Risk 6: Fraudulent Banking Transactions

Compromised customer accounts can be used to perform unauthorized financial transactions.

Risk 7: Data Breach

Sensitive customer information including banking credentials may be exposed.

Risk 8: Compliance Failure

Failure to implement adequate security controls may violate PCI DSS, ISO 27001, GDPR, RBI cybersecurity guidelines, and banking regulations.

4. Threat Identification

Threat	Description	Possible Impact
Credential Stuffing	Attackers use leaked passwords	Customer account takeover
Brute Force Attack	Guessing passwords repeatedly	Unauthorized login
Phishing	Stealing user credentials	Identity theft
API Abuse	Unauthorized API access	Fraudulent transactions
Supply Chain Attack	Compromised third-party libraries	Malware installation
SQL Injection	Manipulating database queries	Customer data theft
Cross Site Scripting (XSS)	Inject malicious scripts	Session hijacking
Cross Site Request Forgery (CSRF)	Unauthorized banking actions	Illegal money transfers

Distributed Denial of Service (DDoS)	Flood banking servers	Service outage
Privilege Escalation	Gain administrator access	Complete system compromise
Ransomware	Encrypt banking databases	Operational shutdown
Insider Threat	Employee misuse	Data leakage

5. Vulnerability Assessment

Vulnerability	Risk Level	Justification
Weak Password Policy	High	Allows brute force and credential attacks
Exposed API Keys	High	Direct access to sensitive services
Outdated Libraries	High	Contain publicly known vulnerabilities
Delayed Security Patching	High	Leaves systems exposed
No Secure Code Reviews	Medium	Coding vulnerabilities remain undetected
Direct Commit to Main Branch	Medium	No validation before deployment
Inconsistent API Authentication	High	Unauthorized access possible
No Continuous Security Testing	Medium	Late vulnerability detection
Poor Log Monitoring	Medium	Incidents remain unnoticed
No Secure Coding Training	Low	Indirect risk but increases future vulnerabilities

---

6. DevOps vs DevSecOps Analysis

DevOps	DevSecOps
Security added at the end	Security integrated throughout SDLC
Focus on rapid delivery	Focus on secure rapid delivery
Manual security testing	Automated security testing
Developers responsible for coding	Everyone shares security responsibility
Limited vulnerability scanning	Continuous vulnerability scanning
Reactive security	Proactive security
Minimal compliance checks	Continuous compliance monitoring
Secrets managed manually	Secure secrets management
No automated policy enforcement	Policy-as-Code implementation
Separate security team	Integrated security collaboration

7. OWASP Security Best Practices

Recommendation 1: Strong Authentication (OWASP Authentication)

Implementation

- Multi-Factor Authentication (MFA)
- Strong password policy
- Password hashing using Argon2 or bcrypt

Benefits

- Prevents unauthorized access
- Protects customer accounts

Recommendation 2: Secure Dependency Management

Implementation

- OWASP Dependency-Check
- Software Composition Analysis (SCA)

Benefits

- Detects vulnerable libraries
- Prevents supply chain attacks

Recommendation 3: Secure API Authentication

Implementation

- OAuth 2.0
- OpenID Connect
- JWT Authentication

Benefits

- Prevents API misuse
- Protects backend services

Recommendation 4: Continuous Security Testing

Implementation

Integrate

- SAST
- DAST
- IAST
- Container Scanning

into CI/CD pipelines.

Benefits

- Detects vulnerabilities early
- Reduces remediation costs

Recommendation 5: Continuous Logging and Monitoring

Implementation

- SIEM Integration
- Centralized logging
- Real-time alerts

Benefits

- Faster incident detection
- Improved forensic investigations

8. Business Impact Analysis

Financial Impact

- Customer fraud losses
- Incident response costs
- Regulatory penalties
- Compensation to affected customers
- Increased cyber insurance premiums

Operational Impact

- Banking service downtime
- Delayed transactions
- Reduced employee productivity
- Increased recovery time

Legal Impact

Failure to comply with banking regulations may result in:

- Regulatory investigations
- Legal proceedings
- Customer lawsuits

Compliance Impact

Possible violations include:

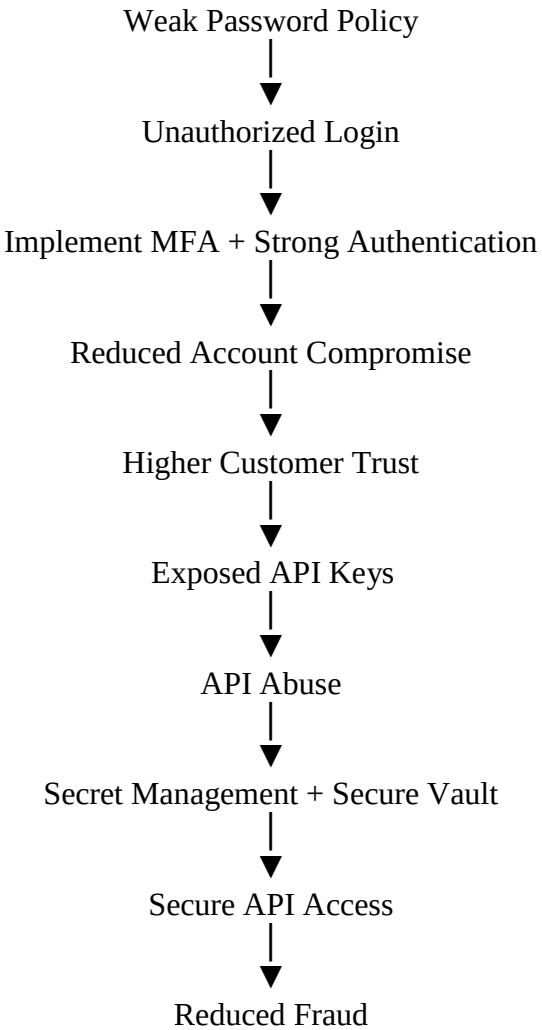
- PCI DSS
- ISO 27001
- GDPR
- RBI Cyber Security Framework

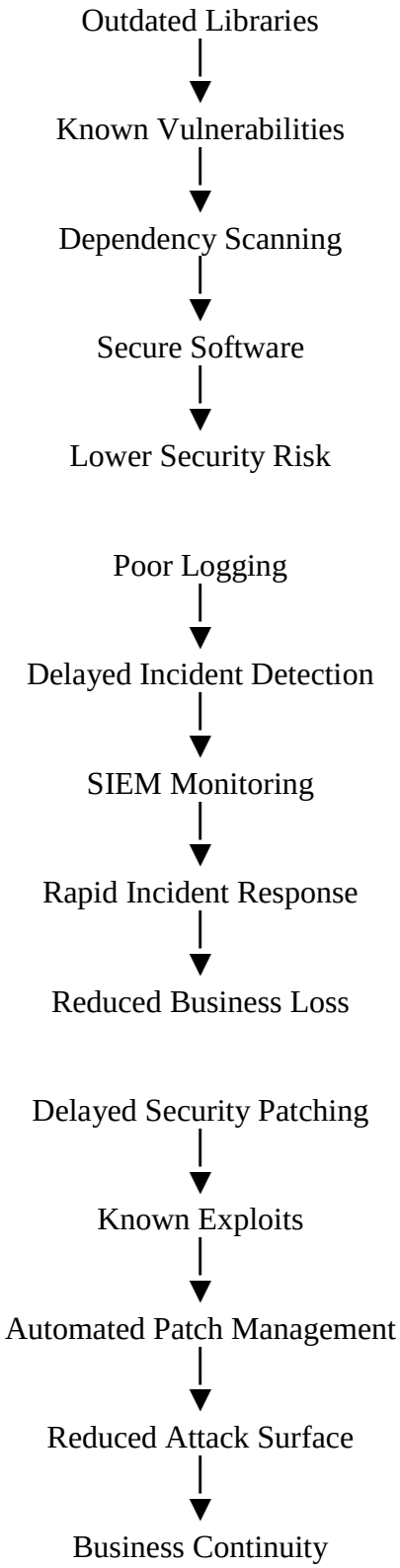
Reputational Impact

Loss of customer confidence may result in:

- Customer migration to competitors
- Reduced market value
- Negative media attention
- Brand damage

9. Concept Map




Outdated Libraries
Known Vulnerabilities
Dependency Scanning
Secure Software
Lower Security Risk
Poor Logging
Delayed Incident Detection
SIEM Monitoring
Rapid Incident Response
Reduced Business Loss
Delayed Security Patching
Known Exploits
Automated Patch Management
Reduced Attack Surface
Business Continuity

10. Conclusion

The assessment identified multiple weaknesses in SecureBank Ltd.'s software development lifecycle, including insecure coding practices, insufficient authentication, outdated software components, delayed patching, and poor monitoring. These issues increase the risk of account compromise, financial fraud, data breaches, and regulatory non-compliance. Adopting DevSecOps practices—such as automated security testing, secure code reviews, dependency management, continuous monitoring, and OWASP-aligned controls—will enable the organization to deliver secure applications while maintaining rapid release cycles. A security-first culture combined with automation and continuous improvement will strengthen resilience against evolving cyber threats.

References

1. OWASP Foundation. (2021). OWASP Top 10: The Ten Most Critical Web Application Security Risks.
2. OWASP Foundation. (2023). OWASP Software Assurance Maturity Model (SAMM).
3. National Institute of Standards and Technology. (2020). NIST Special Publication 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations.
4. National Institute of Standards and Technology. (2018). NIST Secure Software Development Framework (SP 800-218).
5. PCI Security Standards Council. (2022). PCI DSS v4.0 Requirements and Testing Procedures.
6. ISO/IEC. (2022). ISO/IEC 27001:2022 Information Security Management Systems.
7. Kim, G., Humble, J., Debois, P., & Willis, J. (2021). The DevOps Handbook (2nd ed.). IT Revolution Press.
8. Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation. Addison-Wesley.
9. Allspaw, J., & Hammond, P. (2009). 10+ Deploys Per Day: Dev and Ops Cooperation at Flickr.
10. Microsoft. (2024). Microsoft Security Development Lifecycle (SDL) Practices.